

Déploiement avec Portainer - JobbingTrack

[← Déploiement](#) | [← Documentation](#) | [← README principal](#) |  [Navigation](#)

Guide complet pour déployer et gérer JobbingTrack via l'interface graphique Portainer.

Vue d'ensemble

Portainer est une interface web de gestion Docker qui simplifie le déploiement, la surveillance et l'administration de conteneurs Docker.

Avantages Portainer

-  Interface graphique intuitive
-  Gestion des stacks Docker Compose
-  Monitoring en temps réel
-  Gestion des volumes et réseaux
-  Logs centralisés
-  Contrôle d'accès RBAC
-  Templates de déploiement

Installation Portainer

Installation rapide

```
# Créer volume pour données Portainer
docker volume create portainer_data

# Démarrer Portainer
docker run -d \
  -p 8000:8000 \
  -p 9443:9443 \
  --name portainer \
  --restart=always \
  -v /var/run/docker.sock:/var/run/docker.sock \
  -v portainer_data:/data \
  portainer/portainer-ce:latest
```

Accès Interface

1. Ouvrir navigateur: <https://localhost:9443>
2. Créer compte administrateur initial

3. Sélectionner environnement "Local" (Docker)

Déploiement JobbingTrack

Méthode 1: Via Stack Docker Compose

1. Créer un nouveau Stack

1. Menu: **Stacks** → **Add stack**
2. Nom: `jobbingtrack`
3. Build method: **Web editor**

2. Coller docker-compose.yml

```
version: '3.8'

services:
  # Frontend Next.js
  frontend:
    image: jobbingtrack-frontend:latest
    container_name: jobbingtrack-frontend
    ports:
      - "8080:3000"
    environment:
      - NODE_ENV=production
      - NEXT_PUBLIC_API_URL=http://localhost:3000
    restart: unless-stopped
    networks:
      - jobbingtrack-network

  # API Gateway
  api-gateway:
    image: jobbingtrack-api-gateway:latest
    container_name: jobbingtrack-api-gateway
    ports:
      - "3000:3000"
    environment:
      - NODE_ENV=production
      - PORT=3000
    restart: unless-stopped
    networks:
      - jobbingtrack-network
    depends_on:
      - postgres
      - redis

  # PostgreSQL
  postgres:
    image: postgres:16-alpine
```

```

container_name: jobbingtrack-postgres
ports:
  - "5432:5432"
environment:
  - POSTGRES_DB=jobbingtrack
  - POSTGRES_USER=jobbingtrack
  - POSTGRES_PASSWORD=${POSTGRES_PASSWORD}
volumes:
  - postgres_data:/var/lib/postgresql/data
restart: unless-stopped
networks:
  - jobbingtrack-network

# Redis
redis:
  image: redis:7-alpine
  container_name: jobbingtrack-redis
  ports:
    - "6379:6379"
  volumes:
    - redis_data:/data
  restart: unless-stopped
  networks:
    - jobbingtrack-network

volumes:
  postgres_data:
  redis_data:

networks:
  jobbingtrack-network:
    driver: bridge

```

3. Configurer Variables d'Environnement

Dans la section **Environment variables**:

```

POSTGRES_PASSWORD=VotreMotDePasseSécurisé
JWT_SECRET=VotreCléJWTsécurisée
API_RATE_LIMIT=1000

```

4. Déployer le Stack

1. Cliquer **Deploy the stack**
2. Attendre fin du déploiement
3. Vérifier statut des conteneurs

Méthode 2: Via Template Custom

1. Créer Template

Menu: **App Templates** → **Custom Templates** → **Add Custom Template**

Configuration:

- Name: JobbingTrack
- Description: Système de suivi de candidatures
- Platform: Linux
- Type: Stack
- Repository URL: <https://github.com/VotreRepo/JobbingTrack>

2. Déployer depuis Template

1. Menu: **App Templates**
2. Sélectionner JobbingTrack
3. Configurer variables
4. **Deploy the stack**

Méthode 3: Via API Portainer

```
# Obtenir token API
TOKEN=$(curl -X POST http://localhost:9443/api/auth \
  -H "Content-Type: application/json" \
  -d '{"username":"admin","password":"adminpassword"}' \
  | jq -r '.jwt')

# Déployer stack via API
curl -X POST http://localhost:9443/api/stacks \
  -H "Authorization: Bearer $TOKEN" \
  -H "Content-Type: multipart/form-data" \
  -F "Name=jobbingtrack" \
  -F "SwarmID=" \
  -F "StackFileContent=@docker-compose.yml"
```

Voir [Script de déploiement](#) pour automatisation complète.



Configuration Avancée

Gestion des Volumes

1. Menu: **Volumes** → **Add volume**
2. Créer volumes nécessaires:
 - jobbingtrack_postgres_data
 - jobbingtrack_redis_data
 - jobbingtrack_uploads

- `jobbingtrack_logs`

Configuration Réseau

1. Menu: **Networks** → **Add network**
2. Créer réseau: `jobbingtrack-network`
3. Driver: `bridge`
4. Subnet: `172.20.0.0/16` (optionnel)

Secrets et Variables

Pour secrets sensibles (production):

1. Menu: **Secrets** → **Add secret**
2. Créer secrets:
 - `db_password`
 - `jwt_secret`
 - `api_keys`

Référencer dans docker-compose:

```
services:
  api-gateway:
    secrets:
      - db_password
      - jwt_secret
    environment:
      - DB_PASSWORD_FILE=/run/secrets/db_password

secrets:
  db_password:
    external: true
  jwt_secret:
    external: true
```



Monitoring via Portainer

Dashboard Principal

Menu: Home → Sélectionner environnement

Informations affichées:

- Nombre conteneurs (running/stopped)
- Utilisation CPU/RAM
- Images Docker

- Volumes et réseaux
- Stacks déployés

Métriques Conteneur

1. Menu: **Containers**
2. Cliquer sur conteneur
3. Onglet **Stats**

Métriques disponibles:

- CPU Usage
- Memory Usage
- Network I/O
- Block I/O

Logs Temps Réel

1. Menu: **Containers** → Sélectionner conteneur
2. Onglet **Logs**
3. Options:
 - Auto-refresh
 - Timestamp display
 - Ligne count

Alertes (Portainer Business)

Configuration alertes email:

1. Menu: **Settings** → **Notifications**
2. **Add notification**
3. Type: `Webhook` ou `Email`
4. Configurer déclencheurs:
 - Container stopped
 - High CPU/Memory
 - Health check failed

Sécurité

Accès Utilisateurs

1. Menu: **Users** → **Add user**
2. Configurer:
 - Username
 - Password
 - Role (admin/user)
 - Teams (optionnel)

Contrôle d'Accès (RBAC)

Rôles disponibles:

- **Admin:** Accès complet
- **Operator:** Gestion conteneurs/stacks
- **Helpdesk:** Lecture seule + logs
- **User:** Accès limité

Teams et Environnements

1. Menu: **Teams** → **Add team**
2. Assigner environnements
3. Définir permissions

Authentification LDAP/OAuth

1. Menu: **Settings** → **Authentication**
2. Activer LDAP/OAuth
3. Configurer:
 - Server URL
 - Base DN
 - User/Group filters



Mises à jour

Mettre à jour Stack

1. Menu: **Stacks** → Sélectionner `jobbingtrack`
2. **Editor**
3. Modifier configuration
4. **Update the stack**

Options:

- Pull and redeploy
- Prune services
- Re-pull images

Mettre à jour Image

1. Menu: **Images** → Sélectionner image
2. **Pull image**
3. Redéployer conteneurs utilisant l'image

Rolling Update

```
# Via API
curl -X PUT http://localhost:9443/api/stacks/1 \
  -H "Authorization: Bearer $TOKEN" \
  -H "Content-Type: application/json" \
  -d '{
    "StackFileContent": "...",
    "Prune": true,
    "PullImage": true
  }'
```

Backup et Restauration

Backup Stack

1. Menu: **Stacks** → Sélectionner stack
2. **Editor** → Copier configuration
3. Sauvegarder fichier localement

Export Configuration Portainer

```
# Backup données Portainer
docker run --rm \
  -v portainer_data:/data \
  -v $(pwd):/backup \
  alpine tar czf /backup/portainer-backup.tar.gz /data
```

Restauration

```
# Restaurer données
docker run --rm \
  -v portainer_data:/data \
  -v $(pwd):/backup \
  alpine sh -c "cd /data && tar xzf /backup/portainer-backup.tar.gz --strip 1"

# Redémarrer Portainer
docker restart portainer
```

Dépannage

Portainer ne démarre pas

```
# Vérifier logs
docker logs portainer

# Permissions docker.sock
ls -l /var/run/docker.sock

# Redémarrer
docker restart portainer
```

Stack ne se déploie pas

Vérifier:

- Syntaxe docker-compose.yml
- Variables d'environnement définies
- Images disponibles
- Ports non utilisés
- Volumes créés

Conteneurs ne communiquent pas

Vérifier réseau:

1. Menu: **Networks** → Vérifier conteneurs attachés
2. Ping entre conteneurs:

```
docker exec frontend ping api-gateway
```

Performance lente

Actions:

- Vérifier ressources système
- Limiter logs (rotation)
- Nettoyer images inutilisées
- Optimiser volumes



Ressources

- [Scripts Déploiement](#) - Scripts automatisation
- [Guide Production](#) - Configuration production

- [Sécurité](#) - Bonnes pratiques sécurité
- [Monitoring](#) - Monitoring système
- [Documentation Portainer](#) - Documentation officielle

Checklist Déploiement

- ☐ Portainer installé et accessible
 - ☐ Compte admin configuré
 - ☐ Environnement Docker ajouté
 - ☐ Variables d'environnement définies
 - ☐ Secrets créés (production)
 - ☐ docker-compose.yml préparé
 - ☐ Volumes créés
 - ☐ Réseau configuré
 - ☐ Stack déployé
 - ☐ Services démarrés
 - ☐ Health checks OK
 - ☐ Accès frontend/backend testés
 - ☐ Logs consultés
 - ☐ Monitoring configuré
 - ☐ Backup configuré
-

Version: 4.1

Dernière mise à jour: Octobre 2025